

Agentic auto-research in numerical analysis: a coding agent finds, proves and validates a sixth-order Lie-group integrator

Jingquan Wang

Department of Mechanical Engineering
University of Wisconsin–Madison
jingquanw@wisc.edu

Abstract

We report a research program in computational mechanics carried out by a large-language-model coding agent under human direction, from the first prototype to a manuscript with machine-checked proofs, and analyse what made the outcome trustworthy and where it nearly was not. Over four months the agent produced 49 versioned experiments on Lie-group integrators for constrained multibody systems, converged on a three-stage Gauss collocation step with full velocity- and acceleration-level constraint enforcement, and then discovered, while auditing its own manuscript against its own code, that the implemented stage system is exactly reduced Gauss collocation lifted to absolute coordinates. That identity became the central theorem of the resulting paper; the agent formalized it in Lean 4 (57 theorems), designed and ran the experiments, and wrote the manuscript. The process is the subject of this paper: a versioned exploration tree, an evidence ledger of 183 validators that pin every claim to an artefact, a proof sandbox, persistent cross-session memory, explicit command boundaries, and a manuscript pipeline that checks every table entry against its result file. We document ten incidents in which a wrong claim was caught and by what: four by validators or purpose-built experiments, four by the agent’s own cross-checks, two only by the human reading the output. The human typed 36 messages in the final four-day session against 1 108 agent tool calls; the messages that mattered were judgments of taste, scope and fairness rather than instructions. We release the repository, the ledger and the transcript.

1 Introduction

Coding agents built on large language models can now read a code base, run experiments, edit proofs and write prose for days at a time. Whether they can carry a research program is a different question, and the evidence so far comes mostly from short,

benchmark-shaped tasks: an idea to test, a paper to replicate, a Kaggle competition to enter (Lu et al., 2024; Chan et al., 2025; Starace et al., 2025). Real research is longer, its success criteria move, and its characteristic failure is not a crash but a claim that is slightly stronger than what was shown. In numerical analysis this failure mode is acute: a convergence plot can look sixth order for the wrong reason, a comparison with a public code can be unfair in ways that take a domain expert to see, and a theorem can be true of a slightly different method than the one implemented.

This paper is a longitudinal case study of one such program, conducted between May and September 2026 by LLM coding agents under the direction of a single human researcher. The subject of the program was a high-order time integrator for constrained multibody systems modelled in absolute coordinates on a Lie group. Its outcome is a method, Gauss6/FullVA, with an exact algebraic identity at its core, a sixth-order error theorem whose algebra and perturbation lemmas are machine-checked in Lean 4, and a numerical study on a frictional chain and on the four public ASME benchmark mechanisms; that result is reported in a companion manuscript (Wang, 2026). Here we report on the process: what the agent did, how the work was organized so that its claims could be trusted, what went wrong, what caught it, and what the human actually contributed.

We make four contributions.

1. **A documented instance of agentic research with a non-trivial result.** The 49 versioned experiments, the 341 ledger scripts, the Lean development and the manuscript are released with the session transcript of the final phase, so that the process can be inspected rather than described.
2. **A process architecture** (Section 4) of six components that together kept the program

honest: a versioned exploration tree, an evidence ledger of builders and validators with claim gates, a proof sandbox, persistent memory, command and authorization boundaries, and a manuscript pipeline whose validator checks every printed number against its result file.

3. **A trajectory analysis** (Section 5) of how the method emerged: which hypotheses were rejected and why, and how an audit of the manuscript against the code produced the central theorem.
4. **An incident analysis** (Section 6) of ten wrong or misleading claims that arose during the program, classified by what detected them. Validators and purpose-built experiments caught four, the agent’s own cross-checks four, and the human two, both by looking at a figure or a page and saying that it looked wrong; one more needed a human decision after an experiment had failed.

Our reading of the evidence is that the agent was reliable at what can be checked mechanically and unreliable at what cannot: it proved, ran, measured and cross-referenced without fatigue, and it overclaimed, over-hedged and mistook artefacts for results whenever the only check was judgment. The design consequence is not to keep the human in every loop but to build loops that end in an artefact a machine can check, and to route the residual judgments, which are few, to the human.

2 Related work

Automated scientific discovery with LLM agents

End-to-end systems generate ideas, run experiments and write papers (Lu et al., 2024; Yamada et al., 2025; Schmidgall et al., 2025; Jansen et al., 2025), propose hypotheses for wet-lab science (Gottweis et al., 2025), or drive laboratory automation (Boiko et al., 2023; Bran et al., 2024). Their evaluations are typically many short projects judged by reviewers or by a rubric; the systems are the object of study and the science is the test bed. We invert this: one program of research is the object, the science is real (a method that did not exist before, with a proof), and the question is what process made its claims hold up. Program-search systems such as FunSearch and AlphaEvolve (Romera-Paredes et al., 2024; Novikov et al., 2025) discover objects that an automatic evaluator

scores; our setting has no scalar score, and the evaluators (validators, proofs, convergence orders) had to be built as part of the work. Studies of LLM-generated research ideas (Si et al., 2025) measure novelty and feasibility at the idea stage; our concern is the long tail after the idea.

Agents for software engineering and ML engineering

Benchmarks such as SWE-bench, MLE-bench, MLAgentBench, ScienceAgentBench and PaperBench (Jimenez et al., 2024; Chan et al., 2025; Huang et al., 2024; Chen et al., 2025; Starace et al., 2025) measure agents on tasks with a known target and a fixed grader. The coding-agent interface we used (Yang et al., 2024; Anthropic, 2025) is the same kind of tool; the difference is that the target moved as the program learned what was true, and that the grader had to be constructed and then itself frozen when it became stale (Section 4). Agent architectures that interleave reasoning and acting or reflect on failures (Yao et al., 2023; Shinn et al., 2023) describe the inner loop; we describe the outer scaffolding that a months-long program needed around that loop.

Machine-checked mathematics

Language models prove theorems in Lean and other systems (Yang et al., 2023; Song et al., 2024; Xin et al., 2024; Jiang et al., 2023; Trinh et al., 2024). Our use is different in kind: the agent used Lean 4 with Mathlib (de Moura and Ullrich, 2021; The mathlib Community, 2020) as an instrument to check its own analysis of its own code, transcribing the implemented residual into Lean and proving that a specific 132-row system vanishes at a specific point. The formalization did not produce the mathematics; it produced the confidence that the informal argument, written by the same agent, had no gap, and it forced the hypotheses of the theorem to be stated in a form that could be discharged.

Numerical analysis of constrained multibody systems

The domain itself has a long literature on Lie-group integrators and index-3 differential-algebraic equations (Munthe-Kaas, 1998; Brls et al., 2012; Hairer and Wanner, 1996; Kissel et al., 2022; Fang et al., 2023; Chaturvedi et al., 2026); the companion manuscript (Wang, 2026) reviews it.

3 The research problem and its outcome

Problem A mechanism of rigid bodies connected by lower-pair joints (revolute, prismatic, cylindri-

cal, spherical) is described in absolute coordinates by the position and rotation of every body. The equations of motion are an index-3 differential-algebraic system on $(\mathbb{R}^3 \times SO(3))^N$: Newton–Euler balance for each body, algebraic constraints for each joint, Lagrange multipliers for the joint reactions. An integrator has to keep the constraints satisfied at position level and at their first and second time derivatives, keep rotations on the group, and handle joint friction, which is regularized (Brown–McPhee) and can be sharp. Existing Lie-group methods for this class are of order two, or are formulated for ordinary differential equations on the group; a recent time-finite-element method reaches higher order on pendulum problems (Chaturvedi et al., 2026). The program was started with the question whether a method of order higher than four, with a provable order, exists for the full absolute-coordinate index-3 problem.

Outcome The method the program converged on, Gauss6/FullVA, solves one square nonlinear system per step in which three-stage Gauss collocation of the joint coordinates, the joint constraints at position, velocity and acceleration level, and the Newton–Euler balance at every stage appear as rows (132 for the two-body benchmark), and reconstructs the rotational endpoint through the exponential map. The central result is that the reduced Gauss stage, i.e. the collocation solution of the reduced joint-coordinate equations of motion lifted to absolute coordinates, satisfies every one of the 132 rows, and that every root of the non-dynamic rows on the regular branch is such a lift. The method is therefore reduced Gauss collocation executed in absolute coordinates, its sixth order follows from classical theory, and the two things the implementation adds (a velocity-level endpoint closure and an inexact Newton solve) are $O(h^7)$ perturbations with explicit constants. The identity, the perturbation chain and the Butcher conditions are proved in Lean 4 (57 theorems). Numerically the method is sixth order on a frictional two-body chain and on the ASME double pendulum, reproduces the three kinematically driven ASME mechanisms to roundoff, and reaches errors below 10^{-10} where the public absolute-coordinate codes have errors of order one (Wang, 2026).

Why this is a good test of agentic research The result is not a benchmark score. It required choosing among method families, building verifiers that did not exist, recognizing that a coding convention

(joint-coordinate collocation rows projected onto the current axis) was mathematically an identity, formalizing that identity, designing experiments whose references and error measures are defensible, and writing for a journal audience. Each of these is a place where an agent can produce something that looks right and is not.

4 The process architecture

The agent was a general-purpose coding agent (Anthropic, 2025) with file, shell, LaTeX, Lean and git access, working in a repository on the researcher’s machine. Nothing in the agent was specialized to the domain. What was specialized is the set of artefacts the agent was required to produce and consult, summarized in Figure 1. We describe the six components and, for each, the failure it was introduced to prevent.

4.1 Versioned exploration tree

Every hypothesis about the method got its own directory `vNNN_<name>` with a README, the scripts, and a `results/` folder; once a version was superseded it became read-only, and new work started a new version. Forty-nine versions exist (Figure 2). The rule prevents the most common way an agent loses information: editing a working script in place until the earlier state, and the reason it was abandoned, are gone. It also makes negative results first-class: the Lobatto node swap (v024), the projection repair (v025), matrix-free Newton–Krylov (v032) and Jacobian lagging (v033) are all preserved with the numbers that ruled them out, and the manuscript can cite them as things that were tried.

4.2 Evidence ledger: builders, validators, gates

The ledger separates producing evidence from checking it. *Builders* (146 scripts) read result files and write JSON and Markdown records: convergence tables, constraint norms, Newton statistics, claim inventories. *Validators* (183 scripts) read those records and the manuscript and assert relations between them: that a quoted order equals the recorded fit, that a phrase the paper is not allowed to use is absent, that a file the paper cites exists and has the recorded hash, that a gate’s preconditions hold. The top-level validator ran 4 896 checks and the paper-package validator 162 327. *Gates* are named preconditions for stronger claims: the exact-stage-identity gate requires the Lean copy in the

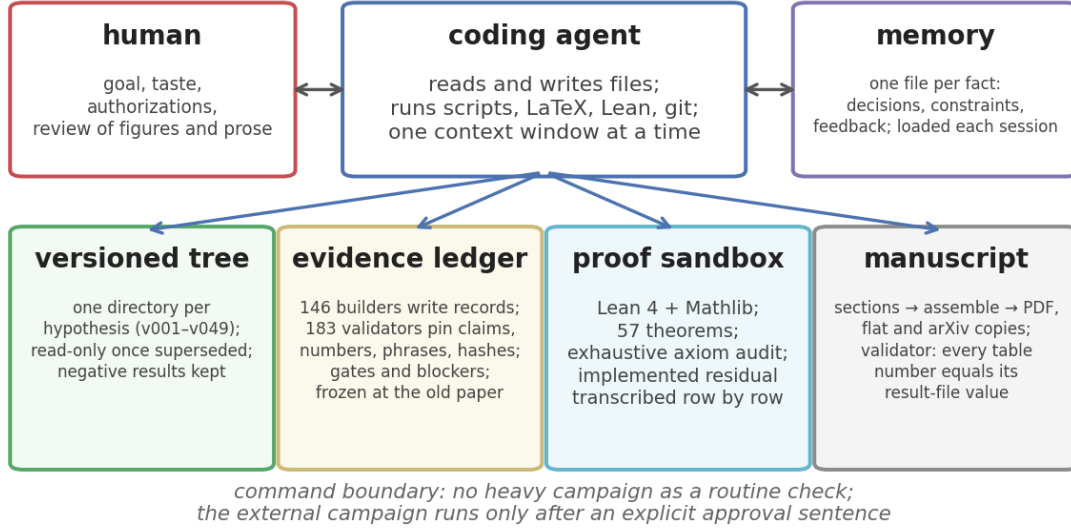


Figure 1: The process. The agent reads and writes five kinds of artefact; the human supplies goals, authorizations and judgments. Every claim in the manuscript must be traceable to a validator-checked record, a Lean theorem or a result file.

repository to be byte-identical to the development and its axiom audit clean; the order-acceptance gate lists which mechanisms may carry a dynamic-order claim. *Blockers* record what would be needed to close an open claim and are kept even when the claim is abandoned. The failure this prevents is claim drift: a diagnostic row becoming a result, a consistency check becoming a proof, a number in the text outliving the run that produced it.

The ledger has a cost: by September it pinned hundreds of phrases of a 13 000-line manuscript and had shaped it into an audit report. When the manuscript was rewritten (Section 5) the ledger was *frozen*, pointed at a snapshot of the old paper as the historical record, and replaced for the new paper by one validator with a different contract (Section 4.6).

4.3 Proof sandbox

The analysis lives in a Lean 4/Mathlib project outside the working tree, with a copy in the repository that a gate keeps synchronized. Fifty-seven theorems cover the row identities of the implemented residual (transcribed into Lean), the perturbation lemmas with explicit constants, the Peano-kernel quadrature bound, the Butcher conditions $B(6)$, $C(3)$, $D(3)$ and the failure of $B(7)$, and the local-to-global transfer. A check script builds the project and runs an exhaustive axiom audit (standard axioms only) and a linter. What is not formalized is stated: Butcher’s order theorem and the classical

Gauss local-error estimate enter as cited results. The proof sandbox changed the paper: the attempt to formalize the theorem’s hypotheses showed that two of them (uniform invertibility of the stage Jacobian, reachability of the solver tolerance) follow from smoothness for small steps, and those derivations became lemmas.

4.4 Persistent memory

The agent’s context is finite and sessions end. A memory directory holds one file per fact, each with a one-line description and a type (project fact, user feedback, reference), and an index loaded at the start of every session. Eleven such files exist; they record, for example, that the benchmark leaves its regular branch at $t \approx 0.605$ and all chain experiments therefore stop at $T = 0.5$, that a particular external campaign may not be rerun without explicit authorization, and that the human judged the earlier manuscript poor and approved a rewrite. The failure this prevents is repetition: without memory, an agent that has learned why an experiment fails will run it again in the next session.

4.5 Command and authorization boundaries

Some commands are expensive or change the meaning of the evidence. The pipeline contract lists them: the full campaign of the method implementation is never run as a routine check, default $h = 10^{-4}$ reruns are forbidden, and the external “source-policy” campaign against the public codes

runs only on explicit human authorization; its driver refuses to start unless a literal approval sentence is passed to it. The one time it ran (September 20) the human’s instruction was two words (“run B4”); the agent supplied the sentence, recorded that it had done so, and the campaign executed 13 commands and promoted 0 of 40 rows, with the reason written down: the public codes’ own reference policy cannot be reproduced by running them harder. The boundary is procedural rather than technical, since the agent can type the sentence; what it buys is a deliberate, recorded step. The failure this prevents is the agent spending hours, or changing the claim state, on its own initiative.

4.6 Manuscript pipeline

The rewritten manuscript is assembled from nine section files by a script; its figures are drawn from result JSON files by another script, so no number is typed; and a validator checks that the LaTeX log is clean, that every number in every result table equals, to the printed precision, a value in the result file the table cites (359 checks), that every Lean theorem named in the paper exists and that the quoted theorem count is the true count, and that the arXiv and flat copies were generated from the current source. The failure this prevents is the one the old ledger had grown to prevent with phrase pins, but at a fraction of the cost and without shaping the prose.

5 Trajectory: how the method was found

Appendix A condenses the exploration into phases (Table 3) and lists every version (Table 4). In outline: Lie-group basics and the choice of Gauss collocation (v001–v005); constrained DAE closure in absolute coordinates, friction, automatic differentiation and quaternions (v006–v013); friction regimes and the rejection of trapezoidal, BDF and Lobatto baselines (v014–v022); the absolute-coordinate revolute DAE and the emergence of the FullVA stage system (v023–v029); sixteen versions of sparse-solver work (v030–v045); validation anchors (v046–v048); and the experiments of the rewritten paper (v049). Three features of the trajectory matter for the argument of this paper.

Negative results were the steering signal Of the 48 exploratory versions, about twenty exist to rule something out. The pattern that produced the method is v023–v029: a full absolute-coordinate revolute DAE (v023) showed endpoint velocity-

level constraint drift; swapping Gauss nodes for Lobatto (v024) did not remove it; projecting the endpoint after the step (v025) removed it but changed the method; adding the velocity-level constraint rows at the stages (v026) and then the acceleration-level rows (v027) removed it without changing the method. The last of these is the FullVA stage system. Its generalization to a second joint (v029) and to non-revolute pairs (v043–v044) then tested that the construction was not specific to one mechanism. Without the preserved negative results the same agent, in a later session, would have retried the projection repair; the memory of why it was wrong is what the versioned tree stores.

Sixteen versions of solver work produced no paper result, and that is recorded v030–v045 investigated sparse Jacobians for the growing stage systems. The sparse structure was found, coloured and cached correctly, and dense automatic differentiation remained faster on every problem tried. The ledger records “sparse structure verified” and forbids “sparse speed superiority”; the companion paper says one sentence about it. An agent optimizing for a paper would have dropped or dressed up this phase; the ledger made the honest outcome the cheap one.

The theorem came from an audit, not from a search By September the manuscript displayed the stage system as body-level collocation rows, while the code solved joint-coordinate collocation rows together with the full constraint hierarchy. The agent was asked to reconcile the two. Working through the row inventory it found that the 132-row residual is zero at the lifted reduced Gauss stage, that the converse holds on the regular branch, and hence that the implemented method *is* reduced Gauss collocation in disguise. This replaced what had been a conditional order argument with an open stage-residual hypothesis by an exact identity plus two explicit perturbations, and it made the sixth order a consequence of classical theory rather than a conjecture supported by slopes. The Lean development was started the same day to check the identity row by row; the two hypotheses that the earlier argument had kept as assumptions were then derived from smoothness. The discovery was not the agent proposing a new idea. It was the agent being required to make two descriptions of the same object agree, and refusing to paper over the difference.

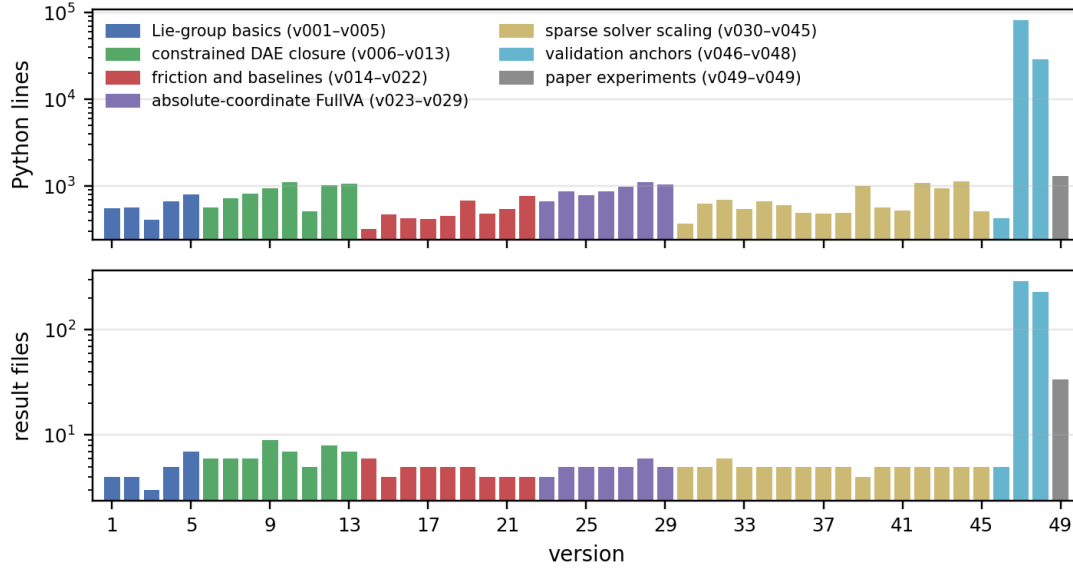


Figure 2: The 49 versioned experiments, coloured by phase, with the size of their code and the number of result files each left behind. The exploratory burst v001–v048 was produced in about three weeks in late May and early June 2026; v046–v048 (validation anchors) and v049 (paper experiments) carry the bulk of the artefacts.

The rewrite The manuscript that the ledger had grown around was 13 000 lines, then 4 000 after a compaction, and read as a catalogue of what was not claimed. The human judged it poor. The agent proposed a full rewrite with a new experiment set, delivered an outline, an experiment plan with costs and an introduction for review, and after approval executed it: seven experiments, three schematic figures, a 44-page manuscript with its arXiv and flat copies, and the new validator, in about four days with 36 human messages (Section 7).

6 Incidents: wrong claims and what caught them

We recorded every case during the final phase in which a statement that had been written or was about to be written turned out to be wrong or misleading. Table 1 lists them with the mechanism that detected each. We group the mechanisms into three kinds: *artefact checks* (a validator or an experiment built to test the statement), *agent cross-checks* (the agent noticing an inconsistency between two things it had itself produced), and *human judgment* (the researcher looking at the output and objecting).

What the artefact checks caught Incidents 3, 5, 7 and 10 were caught by things that fail loudly: a Newton solve that does not converge, a table full of 10^{-14} , a purpose-built measurement, a validator. Incident 7 is the instructive one. The analysis sec-

tion stated that the fixed-tolerance runs satisfy the solver hypothesis with a modest constant; the agent wrote an experiment (E7) whose only purpose was to measure that constant, and the measurement contradicted the text by six orders of magnitude on the finest grid. The text was rewritten to report the measurement. Where experiments are cheap and the norm is to measure a claimed constant rather than argue for it, a class of overclaims becomes data.

What the agent’s cross-checks caught Incidents 2, 6, 8 and 9 were caught because the agent had produced two things that had to agree and did not: four error measures of which one behaved differently, a comparison whose magnitude called for a sanity check at a shorter horizon, a fit whose value contradicted the visible slope, and a sentence in the introduction that the methods section did not support. These are the same habit that produced the central theorem (Section 5): redundancy that the agent is required to reconcile.

What only the human caught Incidents 1 and 4, and the decision in Incident 3, needed a person. Incident 1 is a judgment of quality that no validator expressed and the agent did not have: the manuscript satisfied every check and was bad. Incident 3 was a modelling decision (keep the benchmark as implemented and stop at $T = 0.5$, or change the joint model) that the agent escalated

#	Wrong or misleading statement	Cause	Detected by	Resolution
1	The manuscript was sound but unreadable: hundreds of “diagnostic”, “non-claim” and “boundary” sentences.	Every validator-pinned phrase had been written to satisfy the pin.	H (“the paper quality feels poor”)	Full rewrite; ledger frozen; one number-checking validator.
2	Orientation error converged at order 2.2 while position, velocity and angular velocity converged at order 6.2.	Geodesic angle computed as $2 \arccos(p \cdot q)$, whose resolution floor is 3×10^{-8} rad.	C (one of four quantities disagreed)	Chord formula $2 \arcsin(\ p \mp q\ /2)$; orientation order 6.28.
3	Reference runs on the chain failed at $h = 0.1/256$ over $T = 1$; the plan said $T = 1$ and $T = 20$.	The benchmark’s frozen sliding direction degenerates at $t \approx 0.605$ ($n_1 \cdot a_1 \rightarrow 0$), a modelling limit, not a solver failure.	A (Newton residual 6×10^5) then H (chose to keep the model and stop at $T = 0.5$)	All chain experiments on $[0, 0.5]$; limit stated in the paper.
4	Driven single pendulum: velocity error rose and fell erratically between 10^{-15} and 4×10^{-9} while position showed order 6.	The harness’s Newton predictor is the analytic solution and the stage Jacobian has condition number $10^{10-10^{13}}$; velocities carry amplified roundoff.	H (“why does the top-left panel look strange?”)	Velocity not reported for that mechanism; the panel labelled as a reconstruction check; mechanism explained.
5	The four ASME mechanisms were planned as four convergence tests.	Three of the four are kinematically driven; their motion is fixed by the drive.	A (errors at 10^{-14} for all h)	Only the double pendulum is a dynamics test; the others are consistency rows.
6	The public-code comparison at $T = 3$ showed the public methods with $O(1)$ errors; suspicion of a model mismatch.	The double pendulum amplifies perturbations over $T = 3$.	C (added a $T = 0.1$ alignment run: first-order decrease, same model)	Comparison kept with the alignment check reported.
7	“The h^7 -scaled stopping rule with $c_\eta = 10^4$ covers the reported grids.”	With the fixed tolerance 10^{-11} , the accepted residual over h^7 reaches 10^6 at the finest step.	A (purpose-built experiment E7)	Statement replaced by the measured ratios and a discussion of the pessimistic bound.
8	The single pendulum “converges at order 6 with a floor” was fitted over roundoff points, giving 3.6.	Fit used a zero floor for the analytic reference.	C	Roundoff floor 10^{-15} ; fit 6.0 over the resolved points.
9	“All results generated from a single implementation path.”	Four harnesses assemble the same stage system for four mechanisms.	C (self-review)	Reworded; assemblies listed.
10	After the rewrite the frozen ledger reported stale line counts and lost markers.	Its snapshot records pinned the sizes of scripts that had changed.	A (frozen validators)	Builder sequence rerun; both frozen chains green.

Table 1: Incidents in the final phase. “Detected by” is the mechanism that first raised the problem; A = artefact check (validator or purpose-built experiment), C = agent cross-check, H = human judgment.

with three options and a recommendation to change the model; the human chose to keep it, and the limit was documented. Incident 4 is the one we find most important. The agent had seen the non-monotone velocity curve, written a plausible and wrong explanation (a tolerance artefact of the harness), and moved on. The human’s objection was not technical (“it looks strange”). The investigation it triggered found that the harness starts Newton at the exact solution and that the Jacobian is ill-conditioned by ten to thirteen orders, a fact about the benchmark that no validator could have known to check. Looking at figures is not a formality.

7 Quantitative account

Effort Table 2 summarizes the artefacts. The transcript of the final session (the manuscript rewrite) is profiled in Figure 3: about a third of its tool calls were validators and manuscript builds, a third inspection and editing of files, a tenth Lean, and a twentieth experiments. Earlier phases were not recorded as transcripts and are documented by the version ledger and the dated logs.

What the human said The transcript has 63 user-role entries, but 25 of them are generated by the harness and 2 are shell commands; 36 were typed by the human. They fall into six kinds (Appendix C):

goals (eleven), approvals of the agent’s own proposals (nine), authorizations (three), one decision, judgments (three) and questions (six); the rest were acknowledgments. None contained a technical instruction about the method, the proofs or the experiments. The three judgments produced Incidents 1 and 4 of Table 1 and the layout overhaul of the manuscript; the one decision set the horizon of every chain experiment, and went against the agent’s recommendation.

What the agent did without being asked The agent chose the experiment set and its reference policy (Richardson floor, fit only above $100 \times$ floor), found and fixed the orientation-angle formula, discovered the regular-branch limit and escalated it, added the model-alignment check for the public comparison, wrote the solver-envelope experiment that falsified its own claim, froze the old ledger, and wrote the memory files for the next session. It also produced the three human-caught errors.

Cost of the checks The frozen ledger’s two validation chains take about twenty minutes, the builder sequence that refreshes their snapshots ten, the new manuscript validator two seconds, and the Lean check under a minute once the Mathlib cache is present. The longest single computation was the $h = 10^{-4}$ double-pendulum reference over $T = 3$

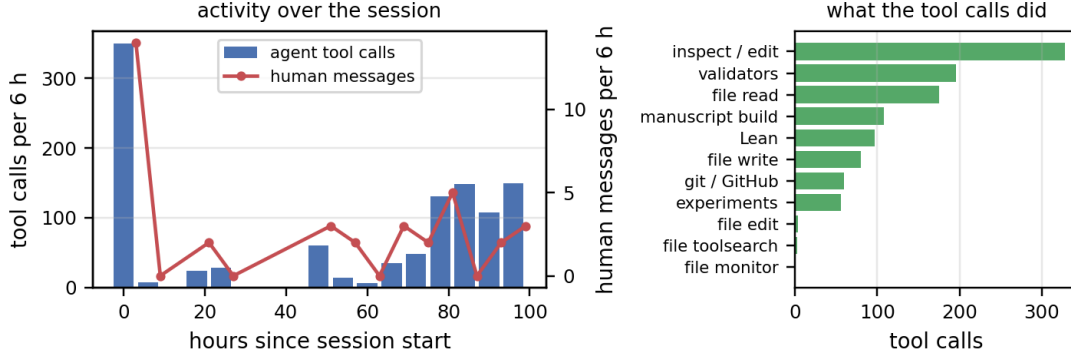


Figure 3: The final session (the manuscript rewrite, September 17–21, 2026), from the transcript: 36 human messages, 1 749 agent turns, 1 108 tool calls. Left: agent tool calls and human messages per six hours. Right: what the tool calls did.

(1.6 h), computed once and cached.

8 Discussion

Where the agent was reliable Everything that ended in an artefact a machine could check. The Lean development has no gaps because Lean would not accept them. The tables match the result files because a validator reads both. The negative results survived because the directory structure made deleting them harder than keeping them.

Where it was not Everything that ended in a sentence. Incident 1 is the clearest: a manuscript that passed 162 327 checks and was, in the researcher’s words, of poor quality; over three months the prose had become a transcript of checks the agent had itself proposed. Incident 4 is the subtler one: the agent saw an anomaly, produced an explanation of the right shape, and accepted it. Both are failures of judgment, not of capability; both were invisible to every mechanism except a person reading.

Design consequences Three follow. First, every claim should be routed toward an artefact check: if a constant is asserted, write the experiment that

measures it (Incident 7); if two descriptions of the same object exist, require them to agree (the theorem). Second, validators should check facts, not prose: the frozen ledger pinned phrases and shaped the paper; its replacement pins numbers to files. Third, the human’s attention should be spent on outputs, not on process: the three human-caught incidents came from reading a page and looking at a figure, and none of the 36 messages that steered the program needed domain expertise beyond recognizing that something looked wrong.

9 Conclusion

A coding agent carried a numerical-analysis research program from a prototype to a manuscript with a new theorem, a machine-checked proof and a defensible numerical study. What made the outcome trustworthy was not the agent’s judgment but artefacts that turned judgments into checks: versioned experiments that could not be edited away, a ledger that tied claims to records, a proof assistant that would not accept a gap, memory across sessions, boundaries on what could be run without asking, and a manuscript whose numbers are read from files. The few residual judgments were the human’s (whether the paper was good, whether a figure looked right, which model to keep), and the practical question for agentic research is then not how to remove the human but how to arrange the work so that these judgments land on the right pages.

Limitations

This is a single case: one program, one domain, one researcher, one family of coding agents, and

Versioned experiments	49
Result files	793
Python lines (all)	356 167
Lean files / lines / theorems	10 / 1 672 / 57
Manuscript	44 pp., 51 refs
Ledger scripts (builders / validators)	341 (146 / 183)
Ledger records (JSON / Markdown)	171 / 186
Ledger checks (top-level / package)	4 896 / 162 327
Manuscript validator checks	359
Memory files	11

Table 2: What the program produced (repository state at the end of the final phase).

no control condition. We cannot say how a different agent, the same agent without the scaffolding, or a human-only effort of the same duration would have fared, and the counts in Table 2 measure output, not quality. The transcript covers only the final phase; the four earlier months are documented by the versioned tree, the dated ledger records and the git history, so the incident list is complete for the final phase and reconstructed for the earlier ones, and incidents that left no trace are invisible to us. The classification of the human messages (Appendix C) was made by the author and the agent from the paraphrased transcript; another reader might draw the category boundaries differently, although the total is fixed. The scaffolding was designed and built by the agent itself and is therefore only as good as the agent’s idea of what to check; the frozen ledger (Section 4) is direct evidence that this idea can drift, and the authorization boundary is procedural rather than technical. Finally, the domain is one in which every claim has a numerical or formal referee (a convergence order, a proof kernel); we do not know how much of the architecture transfers to fields where the checkable artefact is harder to define.

Ethics Statement

The agent chose the experiments, found the identity, wrote the proofs, ran the comparisons and wrote the text of the companion manuscript, and it drafted this paper from the repository and the transcript. The human set the goal, judged the result, made the one modelling decision that was escalated, authorized network access, the GitHub push and one external campaign, and is responsible for the content of both papers; the companion manuscript discloses the use of an AI coding assistant. Under current norms the agent is a tool and the human is the author; the transcript makes the division inspectable. No human subjects were involved, and the only personal data in the released material are the author’s own messages. The public codes compared against are used as published under their licences, and the comparison is reported with its reference policy and horizon so that it can be contested. We see two risks in work of this kind: that agentic research is presented as more autonomous than it was, which is why the human’s contribution is reported message by message, and that checks written by the agent are mistaken for independent review, of which Incident 1 is our own

example.

References

- Anthropic. 2025. Claude Code. <https://docs.anthropic.com/en/docs/claude-code>.
- Daniil A. Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. 2023. Autonomous chemical research with large language models. *Nature*, 624:570–578.
- Andres M. Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D. White, and Philippe Schwaller. 2024. Augmenting large language models with chemistry tools. *Nature Machine Intelligence*, 6:525–535.
- Olivier Brls, Alberto Cardona, and Martin Arnold. 2012. Lie group generalized- α time integration of constrained flexible multibody systems. *Mechanism and Machine Theory*, 48:121–137.
- Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Mdry. 2025. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *International Conference on Learning Representations*.
- Eshan Chaturvedi, Corina Sandu, and Adrian Sandu. 2026. Higher-order integration of index-3 DAE with friction using time finite elements on Lie groups. *Multibody System Dynamics*.
- Ziru Chen, Shijie Chen, Yuting Ning, Qianheng Zhang, Boshi Wang, Botao Yu, Yifei Li, Zeyi Liao, Chen Wei, Zitong Lu, Vishal Dey, Mingyi Xue, Frazier N. Baker, Benjamin Burns, Daniel Adu-Ampratwum, Xuhui Huang, Xia Ning, Song Gao, Yu Su, and Huan Sun. 2025. ScienceAgentBench: Toward rigorous assessment of language agents for data-driven scientific discovery. In *International Conference on Learning Representations*.
- Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, pages 625–635. Springer.
- Luning Fang, Alexandra Kissel, Ruochun Zhang, and Dan Negrut. 2023. On the use of half-implicit numerical integration in multibody dynamics. *Journal of Computational and Nonlinear Dynamics*, 18(1):014501.
- Juraj Gottweis, Wei-Hung Weng, Alexander Daryin, Tao Tu, Anil Palepu, Petar Sirkovic, Artiom Myaskovsky, Felix Weissenberger, Keran Rong, Ryutaro Tanno, et al. 2025. Towards an AI co-scientist. *arXiv preprint arXiv:2502.18864*.
- Ernst Hairer and Gerhard Wanner. 1996. *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems*, 2nd edition. Springer, Berlin.

- Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. 2024. MLAGentBench: Evaluating language agents on machine learning experimentation. In *International Conference on Machine Learning*.
- Peter Jansen, Oyvind Tafjord, Marissa Radensky, Pao Siangliulue, Daniel S. Weld, Bodhisattwa Prasad Majumder, Jonathan Bragg, and Peter Clark. 2025. CodeScientist: End-to-end semi-automated scientific discovery with code-based experimentation. *arXiv preprint arXiv:2503.22708*.
- Albert Q. Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. 2023. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. In *International Conference on Learning Representations*.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*.
- Alexandra Kissel, Dan Negrut, and Jay Taves. 2022. Constrained multibody kinematics and dynamics in absolute coordinates: a discussion of three approaches to representing rigid body rotation. *Journal of Computational and Nonlinear Dynamics*, 17(10):101008.
- Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. 2024. The AI Scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*.
- Hans Munthe-Kaas. 1998. Runge–kutta methods on Lie groups. *BIT Numerical Mathematics*, 38:92–111.
- Alexander Novikov, Ng  n V  , Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, et al. 2025. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*.
- Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. 2024. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475.
- Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Zicheng Liu, and Emad Barsoum. 2025. Agent Laboratory: Using LLM agents as research assistants. *arXiv preprint arXiv:2501.04227*.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36.
- Chenglei Si, Diyi Yang, and Tatsunori Hashimoto. 2025. Can LLMs generate novel research ideas? a large-scale human study with 100+ NLP researchers. In *International Conference on Learning Representations*.
- Peiyang Song, Kaiyu Yang, and Anima Anandkumar. 2024. Towards large language models as copilots for theorem proving in Lean. *arXiv preprint arXiv:2404.12534*.
- Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, Johannes Heidecke, Amelia Glaese, and Tejal Pawardhan. 2025. PaperBench: Evaluating AI’s ability to replicate AI research. *arXiv preprint arXiv:2504.01848*.
- The mathlib Community. 2020. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 367–381.
- Trieu H. Trinh, Yuhuai Wu, Quoc V. Le, He He, and Thang Luong. 2024. Solving olympiad geometry without human demonstrations. *Nature*, 625:476–482.
- Jingquan Wang. 2026. A sixth-order Lie-group Gauss collocation integrator for lower-pair mechanisms with position-, velocity- and acceleration-level constraints. Manuscript, 2026 (repository https://github.com/jqwang2373/integrator_research).
- Huajian Xin, Daya Guo, Zhihong Shao, Zhizhou Ren, Qihao Zhu, Bo Liu, Chong Ruan, Wenda Li, and Xiaodan Liang. 2024. DeepSeek-Prover: Advancing theorem proving in LLMs through large-scale synthetic data. *arXiv preprint arXiv:2405.14333*.
- Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. 2025. The AI Scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *arXiv preprint arXiv:2504.08066*.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37.
- Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. 2023. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems*, volume 36.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.

A The 49 versions

Table 3 condenses the exploration into phases; Table 4 gives one line per version: the method family tested and the role the version plays in the record (its “current role” field in the machine-readable ledger).

B Memory files

The eleven memory files at the end of the program, by their one-line descriptions: where the Lean project lives and why it is kept off the synchronized drive; what the Lean development covers and what it must not be claimed to close; the row-family mismatch between manuscript and code and its resolution ($F(Z_G) = 0$ exactly); the manuscript compaction of September 17 and the builder order; the implemented predictor’s zero angular guesses and their consequence for the Newton lemma; the derived arXiv version and how it is rebuilt; the five-folder repository layout and its path routing; the guarded external campaign of September 20 (approval sentence, 0/40 rows, why it cannot close by execution); the paper rewrite of September 20–21 and its state; the benchmark’s regular-branch limit and the choice $T = 0.5$; and the driven single-pendulum harness whose predictor is the analytic drive. Each file has a “why” and a “how to apply” paragraph; the index is loaded at the start of every session.

C Human messages of the final session

The 36 messages typed by the human (the other 25 user-role entries of the transcript are harness-generated: task notifications, command echoes, compaction summaries and image attachments; 2 more are shell commands typed for the GitHub login), lightly paraphrased from Chinese and grouped. *Goals* (11): can Lean prove the convergence order; install it; run the Lean check and report; continue and look for problems; tidy the Lean development; make an arXiv version in a simple template; reorganize the folders; split the LaTeX out of the ledger; make a clean five-folder layout keeping only the best manuscript; read it again and find what else is wrong; write this paper. *Approvals of the agent’s own proposals* (9): “do as you said” and variants, including the two approvals of the rewrite plan. *Authorizations* (3): any permission including network access; push to GitHub as you see fit; run the external campaign. *Decision* (1): option A for the benchmark horizon, against the agent’s

recommendation. *Judgments* (3): the paper quality feels poor; the PDF does not look good, visualize the pages; why does the top-left panel of Figure 7 look strange. *Questions* (6): what is this doing; what do you think is appropriate; what does that mean; what is still missing; what is the status; what else is there to do. *Other* (3): an acknowledgment, “continue”, and a report that the GitHub login had failed.

Versions	Phase	Kept	Ruled out
v001–v005	Lie-group basics: $SO(3)$ benchmarks, corrected right-action CF4/RKMK4, Yoshida composition, two-stage Gauss–Lie	Gauss collocation as the high-order family; right-action conventions	Yoshida composition (negative substeps unsuitable for friction)
v006–v013	Constrained DAE closure: reduced fixed-pivot DAE, absolute-coordinate Gauss DAE, endpoint constraints inside Newton, friction, AD Jacobian, quaternion transport, three-stage Gauss	absolute coordinates; endpoint constraints in the solve; JAX automatic differentiation; quaternions; six stages	position-only closure; finite-difference Jacobians
v014–v022	Friction regimes and baselines: smoothness sweep, adaptive Gauss6/4, trapezoidal, BDF2 and Lobatto baselines, multiplier-dependent Brown–McPhee friction	Brown–McPhee law through the multipliers; the smooth/sharp split as a stated regime	trapezoidal, BDF2 and Lobatto endpoint methods as accuracy candidates; adaptivity as the main result
v023–v029	Absolute-coordinate revolute DAE: full DAE, Lobatto node swap, projection repair, stage velocity rows, stage acceleration rows, off-axis torque, two joints	<i>stage-level velocity and acceleration constraint rows</i> (“FullVA”), the decisive move	node swap; post-step projection as a fix
v030–v045	Sparse solver scaling: Jacobian sparsity, sparse Newton, matrix-free Krylov, lagging, coloured AD, symbolic patterns, caches, larger topologies, prismatic joints	correctness of sparse patterns; prismatic and skew-axis coverage	every speed claim: dense AD stayed faster
v046–v048	Validation anchors: four ASME mechanisms, two-body cylindrical chain pipeline, cross-paper benchmark harness	the chain as the main benchmark; the public suite as the external one	external superiority claims (source-policy rows 0/40)
v049	Experiments for the rewritten paper	E1–E7	long horizons ($T = 1, T = 20$) on the chain

Table 3: Phases of the exploration. “Kept” is what survived into the final method; “ruled out” is preserved as a versioned negative result.

v	Method family	Role in the record
001	$SO(3)$ kinematics and Euler top	benchmark infrastructure
002	CF4 and RKMK4 (right action)	prescribed-orientation baseline, order four
003	public rA code reproduction	reproducibility anchor for the public suite
004	Yoshida-composed Lie midpoint	conservative baseline; negative substeps unsuitable for friction
005	two-stage Gauss–Lie	smooth conservative mechanics, order four
006	reduced fixed-pivot Gauss DAE	reduced constrained prototype
007	absolute-coordinate Gauss DAE	index-3 consistency lesson
008	endpoint-constrained Gauss DAE	projection-free absolute-coordinate prototype
009	frictional endpoint Gauss4	friction order-reduction evidence
010	JAX AD Jacobian	nonlinear-solve backend
011	S^3 transport operators	bridge to the quaternion formulation
012	quaternion endpoint Gauss4	verified quaternion residual
013	Gauss6 endpoint collocation	first smooth sixth-order candidate
014	Gauss4 vs Gauss6 friction sweep	smoothness decision rule
015	adaptive embedded Gauss6/4	near-nonsmooth regime tool
016	reduced Lie-trapezoidal	ruled out as accuracy candidate
017	reduced Lie-BDF2	ruled out as accuracy candidate
018	reduced Lie-Lobatto endpoint	TFE-style baseline ruled out
019	multiplier-dependent Stribeck friction	AD-friendly friction through multipliers
020	adaptive multiplier friction	sharp-friction regime tool
021	Brown–McPhee multiplier friction	the friction law kept
022	revolute Brown–McPhee Gauss6/4	paper-like revolute friction evidence
023	absolute-coordinate revolute Gauss DAE	full DAE; endpoint drift exposed
024	direct Lobatto full revolute DAE	node swap ruled out
025	projected absolute revolute DAE	projection repair: works, changes the method
026	stage pivot-velocity rows	non-projection closure, velocity level
027	stage velocity + acceleration rows	FullVA : the stage system kept
028	off-axis FullVA revolute	axis-torque stress test
029	double-revolute FullVA	generalizes to two joints
030	Jacobian sparsity diagnosis	motivates sparse solvers
031	sparse Newton	correct; not faster
032	matrix-free Newton–Krylov	ruled out
033	lagged sparse Newton	ruled out
034	coloured-JVP sparse Jacobian	correct structured sparse AD
035	batched coloured JVP	sparse-AD backend
036	block-symbolic sparse pattern	production-style pattern
037	JVP-pruned pattern	pattern discovery
038	pattern cache reuse	cache policy
039	triple-revolute scaling	transfer to larger topology
040	generated block triple pattern	pattern acquisition
041	triple-revolute cache refresh	cache policy
042	skew-axis triple revolute	non-coplanar geometry
043	skew prismatic pair	first non-revolute pair
044	double-prismatic chain	interbody prismatic pair
045	row-coloured VJP sparse AD	sparse-assembly diagnostic
046	four ASME mechanisms	public-suite validation anchor
047	two-body cylindrical chain pipeline	the main benchmark and the accepted implementation
048	cross-paper same-test harness	external comparison scaffold; 0/40 source-policy rows
049	paper experiments E1–E7	the numerical section of the rewritten manuscript

Table 4: Version ledger (condensed from validation/docs/version_ledger.csv).